

SLIIT – Faculty of Computing

BSc (Hons) in Software Engineering

Year 2 Semester 2 – 2026

SE2020 – Web and Mobile Technologies

Individual Assignment – Full Stack Mobile Application

QuickBite – Food Ordering Mobile Application

Project Report

Student: Abisheak S

Student ID: IT21244384

Submission: 2026

1. Problem Statement

Small restaurants and campus canteens around Malabe still take most food orders by phone or at the counter. Staff write orders on paper, customers have no idea whether a dish is still available, and it is common for the kitchen to accept more orders for a popular dish than it can actually prepare. When a dish sells out, customers are phoned back, orders are changed by hand and the bill is recalculated manually, which leads to mistakes and unhappy customers. Customers also cannot see what stage their order is at without calling the restaurant.

QuickBite is a mobile application that solves this for a single restaurant. Customers register, browse a live menu with photos, prices and remaining portions, place an order and follow it through the kitchen (Pending → Preparing → Ready → Completed). The restaurant administrator manages the menu from the same app and moves orders through the workflow.

The key problem the system solves is keeping the menu and the orders **consistent**: an order can never take more portions than are left, a dish is automatically marked unavailable when it sells out, cancelling an order puts the portions back, and the order total is always calculated by the server from the stored prices so it cannot be manipulated from the phone.

1.1 Objectives

- Secure registration and login with bcrypt password hashing and JWT-based sessions.
- Full CRUD for **Menu items** (primary entity), including a photo upload.
- Full CRUD for **Orders** (related entity) that reference menu items, with a status workflow.
- Business rules for stock reservation, automatic sold-out, server-side totals and valid status changes.
- A React Native app with navigation, validated forms and loading / empty / error states, using only data from the hosted API.
- Backend deployed on Render and connected to MongoDB Atlas.

1.2 Users of the system

Role	What they can do
Customer	Register / log in, browse and search the menu, add dishes to a cart, place orders, edit or cancel an order while it is Pending, view order history and status, delete old orders.
Administrator (isAdmin = true)	Create, edit and delete menu items with photos, set stock and availability, view all orders, move orders Pending → Preparing → Ready → Completed, cancel orders.

1.3 Scope boundaries

Following the assignment brief, the system deliberately excludes payment gateways, SMS/e-mail, push notifications, real-time tracking, recommendation features and any web admin panel. Administration is done inside the mobile app. There are exactly two entities besides User: MenuItem and Order.

2. System Architecture

QuickBite is a three-tier client–server system. The React Native app (built with Expo) runs on the phone and talks only to the REST API over HTTPS. The API is a Node.js/Express service deployed on Render; it validates every request, applies the business rules and stores data in a MongoDB Atlas cluster through Mongoose. Secrets are provided to the server as environment variables and are never stored in the repository or in the app.

QuickBite – System Architecture

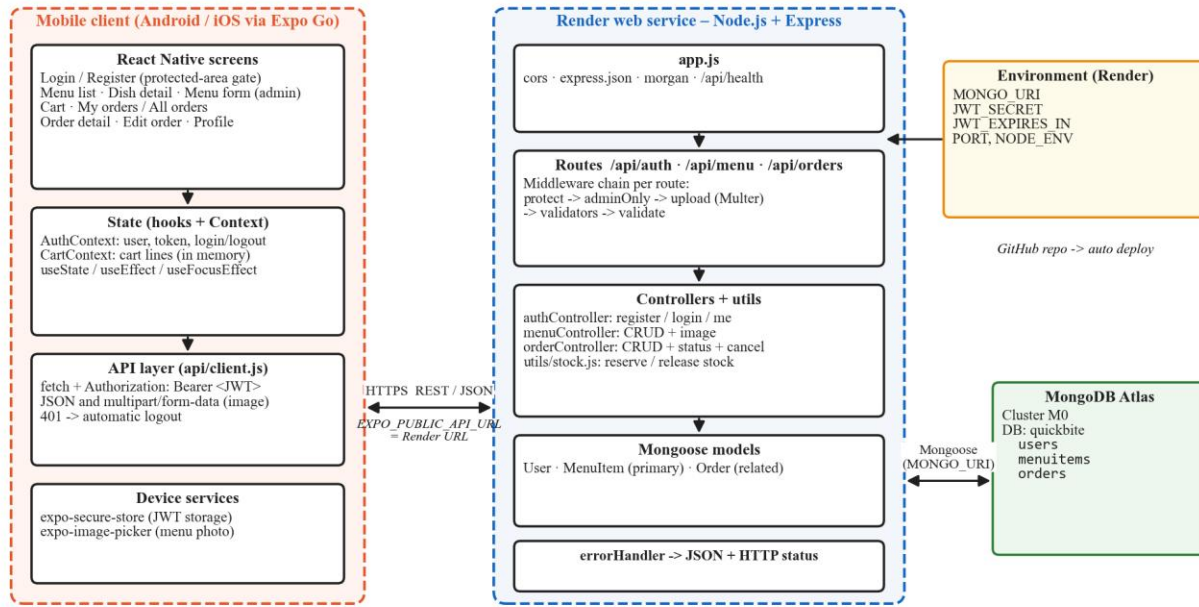


Figure 1 – System architecture

2.1 Technology stack

Layer	Technology	Reason
Mobile	React Native, Expo SDK 57, React Navigation 7	Required by the brief; Expo Go lets the app run on a real phone without a native build.
Device	expo-secure-store, expo-image-picker	Encrypted token storage; choosing a photo for a menu item.
Backend	Node.js, Express 4	Required; simple middleware model for auth, upload and validation.
Data	MongoDB Atlas, Mongoose 8	Required; schemas, validation, references and populate.
Security	bcryptjs, jsonwebtoken	Password hashing and stateless authentication.
Upload / validation	Multer, express-validator	Multipart parsing with type/size limits; input validation before the database.
Hosting	Render (web service), GitHub	Free tier, deploys automatically from the repository.

2.2 Backend structure

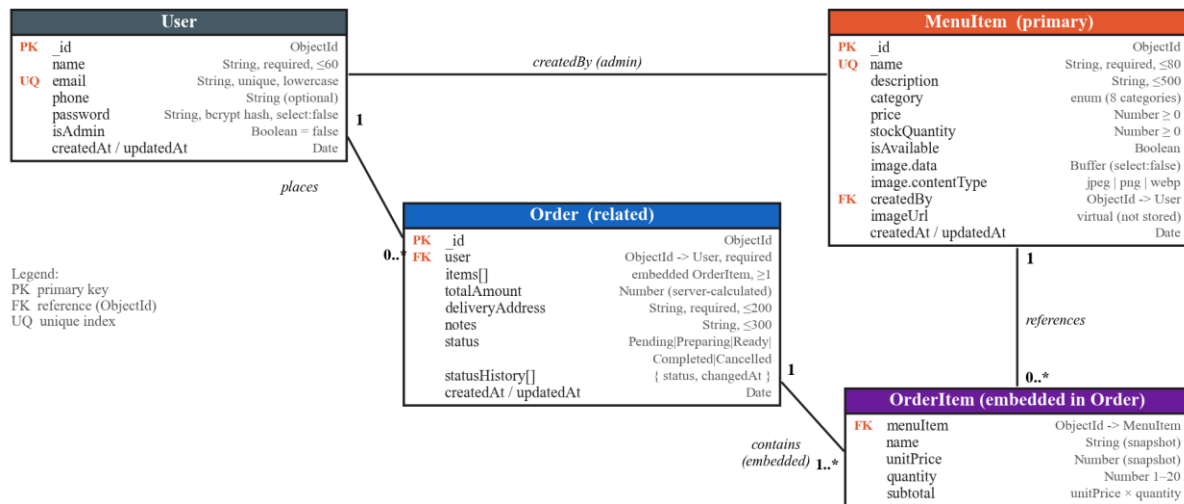
Folder	Responsibility
routes/	Maps each URL + method to a chain of middleware and a controller function.
middleware/	auth.js (protect, adminOnly), upload.js (Multer), validate.js (express-validator result), errorHandler.js (JSON errors + status codes).
validators/	Validation rules for auth, menu and order requests.
controllers/	Request handlers: authController, menuController, orderController.
models/	Mongoose schemas: User, MenuItem, Order.
utils/	ApiError, asyncHandler, stock.js (reserve / release stock), createAdmin script.

3. Database Design

The database has three collections. **MenuItem** is the primary entity. **Order** is the related entity: each order references the User who placed it and, through its embedded items, the MenuItems it contains. Order items are

embedded sub-documents rather than a separate collection because they are never used on their own and are always read together with their order.

QuickBite – Database Schema (MongoDB / Mongoose)



Business rules enforced in code (not by MongoDB):

- Order total = $\Sigma(\text{unitPrice} \times \text{quantity})$ computed on the server; client never sends prices.
- Ordering reserves stock atomically ($\text{stockQuantity} \geq \text{qty}$); multi-item orders are all-or-nothing.
- $\text{stockQuantity} = 0 \rightarrow \text{isAvailable} = \text{false}$; cancelling/deleting a Pending order releases stock.
- Status transitions: Pending $\rightarrow$ Preparing $\rightarrow$ Ready $\rightarrow$ Completed; Cancelled from Pending/Preparing only.
- MenuItem cannot be deleted while referenced by an active (Pending/Preparing/Ready) order.

Figure 2 – Database schema

3.1 Design decisions

- **Reference + snapshot.** Each order line stores the menuItem ObjectId (the relationship) and also copies the name and unit price at order time, so a later price change does not alter an existing order.
- **Server-calculated total.** totalAmount is always $\Sigma(\text{unitPrice} \times \text{quantity})$, computed by the API; the app never sends prices.
- **Status history.** Every status change is appended to statusHistory with a timestamp, so the customer can see when the order moved.
- **Images stored in MongoDB.** Render's file system is ephemeral (wiped on every deploy/restart), so uploaded files cannot be kept on disk. The image bytes are stored in the MenuItem document (limited to 2 MB) and served by GET /api/menu/:id/image; the list endpoints never return the bytes.
- **Password protection.** The password field has select:false so it is never returned by queries, and it is hashed with bcrypt in a pre-save hook.

4. Business Logic

The rules below cannot be enforced by MongoDB alone; they are implemented in orderController.js, menuController.js and utils/stock.js.

Rule	How it is enforced	Response
Cannot order more than is in stock / an unavailable item	reserveStock() uses one atomic findOneAndUpdate with the condition $\{\text{isAvailable}:\text{true}, \text{stockQuantity} \geq \text{qty}\}$ and \$inc: -qty. Two simultaneous orders for the last portion cannot both succeed.	409 Conflict
Multi-item order is all-or-	reserveAll() releases every line already reserved if a later line fails, or if	409

nothing	saving the order fails.	
Sold out ⇒ unavailable	When stock reaches 0 the item's isAvailable is set to false; updating stock to 0 also forces it.	–
Cancel / delete returns stock	Cancelling (customer or admin) or deleting a Pending order calls releaseAll(); an item that had sold out becomes available again.	200
Total recalculated on edit	Editing a Pending order reserves or releases only the difference per item, then recomputes totalAmount.	200 / 409
Valid status workflow	TRANSITIONS map: Pending → Preparing/Cancelled, Preparing → Ready/Cancelled, Ready → Completed. Customers may edit/cancel only while Pending.	409
Protect referenced menu items	A MenuItem in any Pending/Preparing/Ready order cannot be deleted.	409

Core of the stock rule (utils/stock.js):

```
const item = await MenuItem.findOneAndUpdate(
  { _id: menuItemId, isAvailable: true, stockQuantity: { $gte: quantity } },
  { $inc: { stockQuantity: -quantity } },
  { new: true }
);
if (!item) throw new ApiError(409, `Only ${current.stockQuantity} portion(s) left`);
if (item.stockQuantity === 0) { item.isAvailable = false; await item.save(); }
```

5. API Endpoint Table

All endpoints are prefixed with the hosted base URL. “User” means any valid JWT in the Authorization: Bearer header; “Admin” additionally requires isAdmin = true. Errors are returned as JSON: { message, details[] }.

#	Method	Endpoint	Access	Purpose	Success	Errors
1	GET	/api/health	Public	Health check used by Render	200	–
2	POST	/api/auth/register	Public	Create account; returns JWT + user	201	400, 409
3	POST	/api/auth/login	Public	Log in; returns JWT + user	200	400, 401
4	GET	/api/auth/me	User	Current user from token	200	401
5	GET	/api/menu?category=&search=&available=	User	List / filter menu items	200	400, 401
6	GET	/api/menu/categories	User	Allowed categories	200	401
7	GET	/api/menu/:id	User	Read one menu item	200	400, 404
8	GET	/api/menu/:id/image	Public	Image bytes for <Image>	200	404
9	POST	/api/menu (multipart)	Admin	Create item + optional image	201	400, 403, 409
10	PUT	/api/menu/:id (multipart)	Admin	Update fields / replace image	200	400, 403, 404
11	DELETE	/api/menu/:id	Admin	Delete item (blocked if in active order)	200	403, 404, 409
12	POST	/api/orders	User	Place order {items, deliveryAddress, notes}	201	400, 404, 409
13	GET	/api/orders?status=	User	Own orders (admin: all orders)	200	400, 401
14	GET	/api/orders/:id	Owner/Admin	Read one order	200	403, 404
15	PUT	/api/orders/:id	Owner	Edit items/address/notes while Pending	200	403, 409
16	PATCH	/api/orders/:id/cancel	Owner	Cancel Pending order; stock returned	200	403, 409
17	PATCH	/api/orders/:id/status	Admin	Advance status along workflow	200	400, 403, 409
18	DELETE	/api/orders/:id	Owner/Admin	Delete Pending/Cancelled/Completed order	200	403, 404, 409

5.1 Status codes used

Code	Meaning in QuickBite
200 / 201	Success / resource created
400	Validation failed, invalid ObjectId, wrong image type or image larger than 2 MB
401	No token, invalid or expired token, wrong email/password
403	Logged in but not allowed (not admin, or not the order owner)
404	Menu item / order / route not found
409	Business rule conflict: duplicate email/name, out of stock, invalid status change, item in active order
500	Unexpected server error (logged on the server)

6. Mobile Application

The app uses functional components and hooks only. AppNavigator shows either the authentication stack (Login, Register) or the protected area, depending on whether AuthContext holds a user — this is the protected area of the app. The protected area is a native stack containing bottom tabs (Menu, Cart, Orders, Profile) plus detail screens (Dish, Menu form, Order, Edit order). Admins see “Manage menu” and “All orders” and have no cart.

Screen	Purpose	Notable behaviour
Login / Register	Authentication	Client-side validation with messages under each field; server errors shown in a banner.
Menu	List, search and filter by category	Loading, empty and error states; pull to refresh; reloads on focus; “+ Add” for admin.
Dish	Read one item	Customer: quantity stepper limited by stock, add to cart. Admin: edit / delete (409 message shown).
Menu form	Create / update item	Image picker with type/size checks, category chips, numeric validation, availability switch.
Cart	Review and place order	Sends only IDs and quantities; shows 409 stock errors from the server.
Orders	Order history / all orders	Status filter chips, status badges, empty state.
Order	Read one order	Status history; customer edit/cancel while Pending; admin advance status/cancel; delete when allowed.
Edit order	Update order	Change quantities, remove lines, change address/notes; server recalculates total.
Profile	Account	Shows role and connected API URL; log out clears token and cart.

6.1 Token handling

After login the JWT is saved with expo-secure-store and kept in api/client.js, which adds it to every request. On start-up the app validates the stored token with GET /api/auth/me. Any 401 response triggers an automatic logout, returning the user to the Login screen.

[Insert 4–6 screenshots from your phone here: Login, Menu, Dish, Cart, Order detail, Admin menu form.]

7. Deployment Walkthrough

1. **MongoDB Atlas:** created a free M0 cluster, a database user with read/write access, and allowed network access from 0.0.0.0/0 because Render does not have a fixed outbound IP. Copied the connection string and added the database name quickbite.
2. **GitHub:** pushed the repository (backend/ and mobile/ folders). .env files are ignored by .gitignore.
3. **Render:** New → Blueprint → selected the repository. render.yaml creates the web service quickbite-api with root directory backend, build command npm install, start command npm start and health check /api/health.

4. **Environment variables on Render:** MONGO_URI, JWT_SECRET (generated), JWT_EXPIRES_IN=7d, NODE_ENV=production. PORT is provided by Render.
5. **Verification:** opened `https://<service>.onrender.com/api/health` and received `{"status":"ok"}`; the Render log showed “MongoDB connected”.
6. **Admin account:** ran `npm run seed:admin` locally with MONGO_URI pointing at Atlas.
7. **Mobile:** set EXPO_PUBLIC_API_URL in `mobile/.env` to the Render URL, restarted Expo with `npx expo start -c` and tested on a phone using Expo Go.

.env variable	Where	Value
MONGO_URI	Render / backend .env	[redacted]
JWT_SECRET	Render / backend .env	[redacted]
JWT_EXPIRES_IN	Render / backend .env	7d
PORT	Render (automatic)	–
ADMIN_EMAIL, ADMIN_PASSWORD	local backend .env only	[redacted]
EXPO_PUBLIC_API_URL	mobile .env	<code>https://<service>.onrender.com</code>

Live backend URL: `https://<service>.onrender.com` **Repository:** `https://github.com/<user>/quickbite-mobile`

8. Reflection – What Broke and How It Was Fixed

[This page must describe what actually broke for YOU. Deploy the backend, run the app on your phone, and replace or extend the points below with your own real problems, error messages and fixes, in your own words. The points below are pitfalls identified during development and how the code handles them.]

Uploaded images would disappear on Render

Render's file system is ephemeral: files written to an `uploads/` folder are wiped on every deploy or restart. Multer therefore uses `memoryStorage` and the image bytes are stored inside the `MenuItem` document and served from `GET /api/menu/:id/image`. `imageUrl` is a Mongoose virtual with `?v=<updatedAt>` so phones reload the picture after an edit.

Validation and multipart forms

For multipart requests `req.body` is empty until Multer has parsed the form, so running the express-validator rules before Multer makes every field look missing. The upload middleware is therefore placed before the validators. Numbers and booleans arrive as strings in multipart, so the rules convert them with `toFloat()`, `toInt()` and `toBoolean()`.

Two customers buying the last portion

A read-check-save approach lets two simultaneous requests both pass the stock check. A single conditional `findOneAndUpdate` makes check-and-decrement atomic. A multi-item order whose last item is out of stock must not keep the earlier reservations, so `reserveAll()` rolls back the lines already taken. Both cases are covered by the smoke test.

localhost does not work on a phone

On the phone, `localhost` is the phone itself, so an app pointing at `http://localhost:5000` fails with a network error. The URL comes from `EXPO_PUBLIC_API_URL`, which points at the Render service, and the client shows “Cannot reach the server” instead of crashing.

Confirmation dialogs in the web preview

During browser testing, `Alert.alert` with buttons did not show a dialog because `react-native-web` does not support it. A small `confirmAction()` helper uses `window.confirm` on web and `Alert.alert` on phones.

Cold start on Render's free plan

The free service sleeps after 15 minutes and the first request takes about 50 seconds. The app shows loading states, and `/api/health` should be opened before a demo to wake the service.

9. Testing Summary

The API was exercised end-to-end with a scripted smoke test (65 checks) against a test database, covering: registration/login validation and duplicate e-mail, bcrypt hashing, 401 without/with a bad token, 403 for non-admins and non-owners, menu CRUD with image type and size validation, over-stock and unavailable orders (409), all-or-nothing rollback, automatic sold-out and restore on cancel, order edit with stock difference and total recalculation, original price kept after a price change, invalid status transitions (409), and blocked deletion of a menu item in an active order. The mobile flows (register, browse, add to cart, place order, admin create item, advance and cancel order) were tested against the API.

10. Declaration of AI Tool Usage

[Required by Section 6 of the brief. Complete this truthfully: state which AI tools you used, for which parts of the code and documents, and how you checked and learned that code so you can explain every line at the viva.]

Tool	Where it was used	How the output was checked / what I did myself